

NNSDK C/C++ API Guide

Version: v1.0.0

Release Date: 2026-05-31

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 **Amlogic**, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Version	Description
v1.0.0 (2026-05-31)	Initial Release

Contents

1. C API Description

2 Basic Data Structure Definitions

`amlnn_query_cmd`
`amlnn_sdk_version`
`amlnn_input_output_num`
`amlnn_tensor_format`
`amlnn_tensor_type`
`amlnn_tensor_attr`
`amlnn_input_range`
`amlnn_profiling_data`
`amlnn_perf_run`
`amlnn_npu_core_status`
`amlnn_input`
`amlnn_output`
`amlnn_backend_type`
`amlnn_model_task_priority`
`amlnn_init_config`
`amlnn_run_config`
`amlnn_dma_memory`
`amlnn_ext_mem_access_mode`
`amlnn_ext_buffer_type`
`amlnn_ext_io_type`
`amlnn_dma_config`
`amlnn_softop_type`
`amlnn_softop_acc_type`
`amlnn_softop_opt_request`
`amlnn_core_mask_policy`
`amlnn_core_mask_config`

3 Basic API Introduction

`amlnn_init`
`amlnn_query`
`amlnn_inputs_set`
`amlnn_run`
`amlnn_outputs_get`
`amlnn_destroy`
`amlnn_alloc_dma_mem`
`amlnn_bind_dma_mem`
`amlnn_unbind_dma_mem`
`amlnn_free_dma_mem`
`amlnn_set_softop_opt`
`amlnn_set_input_shapes`
`amlnn_set_core_mask`

4 Error Return Codes

1. C API Description

The C API has been upgraded to the second generation. Data structures and APIs are defined in `nnsdk2.h`. To learn about or continue using the first-generation C API, refer to NNSDK1 Documents.

2 Basic Data Structure Definitions

This section lists all data structures in the same order as `nnsdk2.h`.

`amlnn_query_cmd`

Enumeration type used to select commands for `amlnn_query`.

Member	Description
<code>AMLNN_QUERY_SDK_VERSION</code>	Queries the SDK and driver versions
<code>AMLNN_QUERY_IN_OUT_NUM</code>	Queries the number of inputs and outputs
<code>AMLNN_QUERY_INPUT_ATTR</code>	Query Input Tensor Attributes
<code>AMLNN_QUERY_OUTPUT_ATTR</code>	Query Output Tensor Attributes
<code>AMLNN_QUERY_INPUT_DYNAMIC_RANGE</code>	Queries the dynamic shape range of input tensors
<code>AMLNN_QUERY_CURRENT_INPUT_ATTR</code>	Queries the current input shape (valid only for dynamic models)
<code>AMLNN_QUERY_CURRENT_OUTPUT_ATTR</code>	Queries the current output shape (valid only for dynamic models)
<code>AMLNN_QUERY_PERF_DETAIL</code>	Queries profiling data and triggers output of the performance detail text file; when calling <code>amlnn_init</code> , <code>enable_perf</code> in <code>amlnn_init_config</code> must be enabled, <code>perf_detail_path</code> must be set, and this command must be called after <code>amlnn_run</code> ; ADLA NPU only
<code>AMLNN_QUERY_PERF_RUN</code>	Queries run duration; must be called after <code>amlnn_run</code>
<code>AMLNN_QUERY_MODEL_SOFTOP</code>	Queries software operator information of the model (TODO)
<code>AMLNN_QUERY_NPU_CORE_STATUS</code>	Queries NPU core status

amlnn_sdk_version

The `amlnn_sdk_version` structure represents SDK version information and is defined as follows:

Member	Data Type	Description
sdk2_api_version	char[]	AML NNSDK2 API version
adla_kmd_version	char[]	ADLA driver version (NPU)
delegate_version	char[]	Delegate version (GPU/CPU)

amlnn_input_output_num

The `amlnn_input_output_num` structure represents the number of model inputs and outputs and is defined as follows:

Member	Data Type	Description
n_input	uint32_t	Number of inputs
n_output	uint32_t	Number of outputs

amlnn_tensor_format

Enumeration type that represents tensor data layout format.

Member	Description
AMLNN_TENSOR_NHWC	NHWC format
AMLNN_TENSOR_NCHW	NCHW format

amlnn_tensor_type

Enumeration type that represents tensor data type.

Member	Description
AMLNN_TENSOR_FLOAT32	FP32 type
AMLNN_TENSOR_FLOAT16	FP16 type
AMLNN_TENSOR_INT8	INT8 type
AMLNN_TENSOR_UINT8	UINT8 type
AMLNN_TENSOR_INT16	INT16 type
AMLNN_TENSOR_UINT16	UINT16 type
AMLNN_TENSOR_INT32	INT32 type
AMLNN_TENSOR_UINT32	UINT32 type
AMLNN_TENSOR_INT64	INT64 type

Member	Description
AMLNN_TENSOR_UINT64	UINT64 type
AMLNN_TENSOR_BOOL	BOOL type
AMLNN_TENSOR_INT4	INT4 type
AMLNN_TENSOR_BFLOAT16	BF16 type

amlnn_tensor_attr

The `amlnn_tensor_attr` structure represents model input/output tensor attributes and is defined as follows:

Member	Data Type	Description
index	uint32_t	Input parameter: input/output tensor index
n_dims	uint32_t	Number of dimensions
dims	uint32_t[]	Dimension array
name	char[]	Tensor name
n_elems	uint32_t	Number of elements
size	uint32_t	Tensor size in bytes
fmt	amlnn_tensor_format	Tensor data layout format
type	amlnn_tensor_type	Tensor data type
zp	int32_t	Zero-point parameter
scale	float	Scale parameter

amlnn_input_range

The `amlnn_input_range` structure represents the supported shape list information of one input. It includes the input index, number of supported shapes, data layout format, name, and shape list, and is defined as follows:

Member	Data Type	Description
index	uint32_t	Input parameter: input tensor index
shape_number	uint32_t	Number of dynamic shapes
fmt	amlnn_tensor_format	Tensor data layout format
name	char[]	Tensor name
dyn_range	uint32_t[] []	Dynamic input dimension range
n_dims	uint32_t	Number of dimensions

amlnn_profiling_data

The `amlnn_profiling_data` structure represents performance profiling data (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
us_elapsed_in_hw_op	uint64_t	Hardware operator execution duration
us_elapsed_in_sw_op	uint64_t	Software operator execution duration
memory_usage_bytes	uint64_t	Memory usage
dram_read_bytes	uint64_t	DDR read traffic (bandwidth)
dram_write_bytes	uint64_t	DDR write traffic (bandwidth)
sram_read_bytes	uint64_t	SRAM read traffic (bandwidth)
sram_write_bytes	uint64_t	SRAM write traffic (bandwidth)
axi_freq_cur	uint64_t	Current NPU AXI frequency
core_freq_cur	uint64_t	Current NPU core frequency
mem_allocated_base	uint64_t	Base memory occupied by the driver
mem_allocated_umd	uint64_t	Memory occupied by the current model
mem_pool_size	uint64_t	Address space size supported by the driver
mem_pool_used	uint64_t	Total memory occupied by the driver (mem_allocated_base + mem_allocated_umd(modelA) + ...)
reserved	uint8_t[]	Reserved

amlnn_perf_run

The `amlnn_perf_run` structure represents timing statistics for a single run and is defined as follows:

Member	Data Type	Description
run_duration	int64_t	Actual inference duration (us)

amlnn_npu_core_status

The `amlnn_npu_core_status` structure represents NPU core status (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
core_num	uint32_t	Number of NPU cores on the current device

Member	Data Type	Description
core_mask	uint64_t	NPU core mask of the current device, represented by bits
idle_mask	uint64_t	Mask of NPU cores currently idle
busy_mask	uint64_t	Mask of NPU cores currently busy
reserved	uint8_t[]	Reserved

amlnn_input

The `amlnn_input` structure represents one data input of the model and is passed as a parameter to `amlnn_inputs_set`. It is defined as follows:

Member	Data Type	Description
index	uint32_t	Input index
buf	void*	Input data buffer pointer
size	uint32_t	Input data buffer size in bytes

amlnn_output

The `amlnn_output` structure represents one data output of the model and is passed as a parameter to `amlnn_outputs_get`. After the function runs, the structure object is populated. It is defined as follows:

Member	Data Type	Description
want_float	uint8_t	Whether float output is required
index	uint32_t	Output index
buf	void*	Output data buffer pointer
size	uint32_t	Output data buffer size in bytes

amlnn_backend_type

Enumeration type used to specify the backend type.

Member	Description
AMLNN_BACKEND_ADLA_NPU	Runs an ADLA model (ADLA NPU)
AMLNN_BACKEND_TF_DELEGATE_GPU	Runs a TFLite model (GPU)
AMLNN_BACKEND_TF_DELEGATE_CPU	Runs a TFLite model (CPU)

amlnn_model_task_priority

Enumeration type used to specify model task priority.

Member	Description
AMLNN_MODEL_TASK_PRIOR_MEDIUM	Model task has medium priority
AMLNN_MODEL_TASK_PRIOR_LOW	Model task has low priority
AMLNN_MODEL_TASK_PRIOR_HIGH	Model task has high priority

amlnn_init_config

The `amlnn_init_config` structure represents the initialization configuration for `amlnn_init` (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
backend_type	amlnn_backend_type	Backend type
flags	uint32_t	Initialization flags (TODO)
task_priority	amlnn_model_task_priority	Model task priority
timeout_ms	uint32_t	Invoke timeout
enable_perf	uint32_t	Whether to collect model performance data
perf_detail_path	const char*	Text output path for performance data
reserved	uint8_t[]	Reserved

amlnn_run_config

The `amlnn_run_config` structure represents the runtime configuration for `amlnn_run` and is defined as follows:

Member	Data Type	Description
reserved	uint8_t[]	Reserved

amlnn_dma_memory

The `amlnn_dma_memory` structure represents DMA memory information (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
handle	void*	Internal memory handle
viraddr	void*	Virtual address
phyaddr	void*	Physical address

Member	Data Type	Description
index	uint32_t	Memory index identifier
size	uint32_t	Allocated memory size

amlnn_ext_mem_access_mode

Enumeration type that represents DMA access mode (ADLA NPU only).

Member	Description
AMLNN_EXT_MEM_ACCESS_READ_WRITE	CPU readable and writable
AMLNN_EXT_MEM_ACCESS_READ_ONLY	CPU read-only (TODO)
AMLNN_EXT_MEM_ACCESS_WRITE_ONLY	CPU write-only (TODO)

amlnn_ext_buffer_type

Enumeration type that represents DMA address type (ADLA NPU only).

Member	Description
AMLNN_EXT_BUFFER_VIRTUAL_ADDR	Virtual address
AMLNN_EXT_BUFFER_PHYS_ADDR	Physical address
AMLNN_EXT_BUFFER_SECURE_PHYS_ADDR	Secure physical address

amlnn_ext_io_type

Enumeration type that represents DMA binding direction (ADLA NPU only).

Member	Description
AMLNN_EXT_IO_INPUT	Bind as input
AMLNN_EXT_IO_OUTPUT	Bind as output

amlnn_dma_config

The `amlnn_dma_config` structure represents DMA configuration (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
cacheable	uint32_t	Enable cache to speed up CPU access
access_mode	amlnn_ext_mem_access_mode	CPU access type
buffer_type	amlnn_ext_buffer_type	Buffer address type
io_type	amlnn_ext_io_type	Input or output

amlnn_softop_type

Enumeration type that represents software operator type (ADLA NPU only). This enum is long, so the table below lists only a few examples. See `nnsdk2.h` for the complete list.

Member	Description
AMLNN_Add	Add type
AMLNN_Conv2d	Conv2d type
AMLNN_DepthwiseConv2d	DepthwiseConv2d type
AMLNN_Relu	Relu type
AMLNN_Softmax	Softmax type
AMLNN_Unknown	Unknown type

amlnn_softop_acc_type

Enumeration type that represents software operator acceleration type (ADLA NPU only).

Member	Description
AMLNN_SOFTOP_ACC_OPENMP	OpenMP acceleration
AMLNN_SOFTOP_ACC_VULKAN	Vulkan acceleration (TODO)
AMLNN_SOFTOP_ACC_OPENCL	OpenCL acceleration (TODO)

amlnn_softop_opt_request

The `amlnn_softop_opt_request` structure represents software operator acceleration configuration (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
softop_type	amlnn_softop_type	Software operator type
acc_type	amlnn_softop_acc_type	Software operator acceleration type

amlnn_core_mask_policy

Enumeration type that represents the policy for setting the NPU core mask (ADLA NPU only).

Member	Description
AMLNN_CORE_MASK_AUTO	The system automatically schedules and selects the NPU core to execute the task
AMLNN_CORE_MASK_BEST_EFFORT	Attempts to use the user-requested NPU core, but availability is not guaranteed

amlnn_core_mask_config

The `amlnn_core_mask_config` structure represents NPU core mask configuration information (ADLA NPU only) and is defined as follows:

Member	Data Type	Description
requested_mask	uint64_t	Requested NPU core mask
policy	amlnn_core_mask_policy	Selected NPU core mask policy
reserved	uint8_t[]	Reserved

3 Basic API Introduction

amlnn_init

The `amlnn_init` initialization function creates an `amlnn` context object, loads an ADLA/TFLite model according to the selected backend type, and performs specific initialization actions according to the `amlnn_init_config` structure.

API	amlnn_init
Function	Initializes the <code>amlnn</code> context handle
Parameters	<code>void** context</code> : Double pointer to the context handle
	<code>void* model</code> : Binary model data or model path (if the selected backend is not ADLA NPU, only a model path is supported) When <code>size</code> is greater than 0, <code>model</code> represents binary data When <code>size</code> equals 0, <code>model</code> represents a model path
	<code>uint32_t size</code> : When <code>model</code> is binary data, this indicates model size; when <code>model</code> is a path, set it to 0
	<code>amlnn_init_config* init_config</code> : Initialization configuration parameters
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
void *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));
ret = amlnn_init(&ctx, (void *)model_path, 0, &init_config);
```

amlnn_query

The `amlnn_query` function can query information such as SDK version, model input/output information, profiling data, layer-by-layer performance detail text files, and total model inference time.

API	amlnn_query
Function	Queries model and SDK related information
Parameters	<code>void* context</code> : Context handle pointer
	<code>amlnn_query_cmd</code> : Query command
	<code>void* info</code> : Structure variable used to store the returned result
	<code>uint32_t size</code> : Size of the structure variable corresponding to <code>info</code>
Return Value	int error code (see AMLNN API Error Return Codes)

The query commands supported by the current SDK are shown in the table below:

Query command	Returned Structure	Function
AMLNN_QUERY_SDK_VERSION	amlnn_sdk_version	Query SDK Version
AMLNN_QUERY_IN_OUT_NUM	amlnn_input_output_num	Query the Number of Input and Output Tensors
AMLNN_QUERY_INPUT_ATTR	amlnn_tensor_attr	Query Input Tensor Attributes
AMLNN_QUERY_OUTPUT_ATTR	amlnn_tensor_attr	Query Output Tensor Attributes
AMLNN_QUERY_INPUT_DYNAMIC_RANGE	amlnn_input_range	When using a model that supports dynamic shapes, queries the number and list of supported input shapes, the data layout corresponding to each shape, names, and other information
AMLNN_QUERY_CURRENT_INPUT_ATTR	amlnn_tensor_attr	When using a model that supports dynamic shapes, queries the input attributes used by the current inference
AMLNN_QUERY_CURRENT_OUTPUT_ATTR	amlnn_tensor_attr	When using a model that supports dynamic shapes, queries the output attributes used by the current inference
AMLNN_QUERY_PERF_DETAIL	amlnn_profiling_data	Queries profiling data and exports a layer-by-layer performance detail text file. This takes effect only when <code>enable_perf</code> in <code>amlnn_init_config</code> is enabled and <code>perf_detail_path</code> is set when calling <code>amlnn_init</code>
AMLNN_QUERY_PERF_RUN	amlnn_perf_run	Queries model inference duration (excluding input/output setup), in microseconds

Detailed usage of each command is as follows:

1. Query SDK Version

Passing the `AMLNN_QUERY_SDK_VERSION` command queries SDK version information. Create an `amlnn_sdk_version` structure object first. Example code:

```
amlnn_sdk_version sdk_ver;  
ret = amlnn_query(NULL, AMLNN_QUERY_SDK_VERSION, &sdk_ver, sizeof(sdk_ver));  
printf("verion info: sdk2-[%s], adla-[%s], delegate-[%s]\n",  
sdk_ver.sdk2_api_version, sdk_ver.adla_kmd_version,  
sdk_ver.delegate_version);
```

2. Query the Number of Input and Output Tensors

After `amlnn_init` has been called, passing the `AMLNN_QUERY_IN_OUT_NUM` command queries the number of model input and output tensors. Create an `amlnn_input_output_num` structure object first. Example code:

```
amlnn_input_output_num io_num;
ret = amlnn_query(ctx, AMLNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
printf("model input num: %d, output num: %d\n", io_num.n_input,
io_num.n_output);
```

3. Query Input Tensor Attributes

After `amlnn_init` has been called, passing the `AMLNN_QUERY_INPUT_ATTR` command queries model input tensor attributes. Create an `amlnn_tensor_attr` structure object first. Example code:

```
amlnn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(amlnn_tensor_attr));
}
```

4. Query Output Tensor Attributes

After `amlnn_init` has been called, passing the `AMLNN_QUERY_OUTPUT_ATTR` command queries model output tensor attributes. Create an `amlnn_tensor_attr` structure object first. Example code:

```
amlnn_tensor_attr output_attrs[io_num.n_output];
memset(output_attrs, 0, sizeof(output_attrs));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(amlnn_tensor_attr));
}
```

5. Query Dynamic Input Shape Information Supported by the Model

After `amlnn_init` has been called, passing the `AMLNN_QUERY_INPUT_DYNAMIC_RANGE` command queries the input shape information supported by the model, including the number of input shapes, input shape list, layout and name corresponding to each input shape, and other information. Create an `amlnn_input_range` structure object first. Example code:

```

amlnn_input_range shape_range[io_num.n_input];
memset(shape_range, 0, io_num.n_input * sizeof(amlnn_input_range));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    shape_range[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_DYNAMIC_RANGE, &shape_range[i],
sizeof(amlnn_input_range));
}

```

6. Query the Input Dynamic Shape Currently Used by the Model

After `amlnn_set_input_shapes` has been called, passing the `AMLNN_QUERY_CURRENT_INPUT_ATTR` command queries the input attribute information currently used by the model. Create an `amlnn_tensor_attr` structure object first. Example code:

```

amlnn_tensor_attr cur_input_attrs[io_num.n_input];
memset(cur_input_attrs, 0, io_num.n_input * sizeof(amlnn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    cur_input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_CURRENT_INPUT_ATTR, &
(cur_input_attrs[i]), sizeof(amlnn_tensor_attr));
}

```

7. Query the Output Dynamic Shape Currently Used by the Model

After `amlnn_set_input_shapes` has been called, passing the `AMLNN_QUERY_CURRENT_OUTPUT_ATTR` command queries the output attribute information currently used by the model. Create an `amlnn_tensor_attr` structure first. Example code:

```

amlnn_tensor_attr cur_output_attrs[io_num.n_output];
memset(cur_output_attrs, 0, io_num.n_output * sizeof(amlnn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    cur_output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_CURRENT_OUTPUT_ATTR, &
(cur_output_attrs[i]), sizeof(amlnn_tensor_attr));
}

```

8. Query Profiling Data and Export a Layer-by-Layer Performance Detail Text File

After `amlnn_run` has been called, passing `AMLNN_QUERY_PERF_DETAIL` to `amlnn_query` obtains profiling data and exports a layer-by-layer performance detail text file. To use this command, enable `enable_perf` in the `amlnn_init_config` structure of the `amlnn_init` API and set `perf_detail_path` as the output path for the text file. Example code:

```

void *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));
init_config.enable_perf = 1;
init_config.perf_detail_path = "/data/nn";
ret = amlnn_init(&ctx, (void *)model, model_len, &init_config);
...
ret = amlnn_run(ctx, NULL);
...
amlnn_profiling_data profiling_data;
memset(&profiling_data, 0, sizeof(amlnn_profiling_data));
ret = amlnn_query(ctx, AMLNN_QUERY_PERF_DETAIL, &profiling_data,
sizeof(profiling_data));
// After the call completes, `profile_info.txt` is generated under
`/data/nn`, recording layer-by-layer performance details for this inference,
including duration, bandwidth, and other data.

```

9. Query Total Model Inference Duration

After `amlnn_run` has been called, passing `AMLNN_QUERY_PERF_RUN` to `amlnn_query` queries the model inference duration (excluding input/output setup), in microseconds. Example code:

```

amlnn_perf_run perf_run;
memset(&perf_run, 0, sizeof(amlnn_perf_run));
ret = amlnn_query(ctx, AMLNN_QUERY_PERF_RUN, &perf_run, sizeof(perf_run));

```

amlnn_inputs_set

The `amlnn_inputs_set` function sets model input data. This function supports multiple inputs. Each input is an `amlnn_input` structure object, which must be configured by the user before being passed in.

API	amlnn_inputs_set
Function	Sets model input data
Parameters	void* context: Context handle pointer
	uint32_t n_inputs: Number of input data items
	amlnn_input inputs[]: Input data array; each array element is an <code>amlnn_input</code> structure object
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```

amlnn_input inputs[1];
memset(inputs, 0, sizeof(inputs));
inputs[0].index = 0;
inputs[0].buf = input_data; // Preprocessing must be completed externally,
including normalization, quantization, data type matching, and data layout format
matching. The SDK directly copies this data block to the NPU buffer internally.
inputs[0].size = input_size; // Input data size in bytes
ret = amlnn_inputs_set(ctx, io_num.n_input, inputs);

```

amlnn_run

The `amlnn_run` function executes one model inference. Before calling it, input data must be set using `amlnn_inputs_set` or the DMA APIs.

API	amlnn_run
Function	Executes one model inference
Parameters	void* context: Context handle pointer
	amlnn_run_config* run_config: Inference configuration parameters; currently unused, pass NULL
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
ret = amlnn_run(ctx, NULL);
```

amlnn_outputs_get

The `amlnn_outputs_get` function obtains model inference output data. It can obtain multiple outputs at once. Each output is an `amlnn_output` structure object, and each object must be created and configured before calling the function.

API	amlnn_outputs_get
Function	Gets model inference output
Parameters	void* context: Context handle pointer
	uint32_t n_outputs: Number of output data items
	amlnn_output outputs[]: Output data array; each array element is an <code>amlnn_output</code> structure object that represents one model output
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
amlnn_output outputs[1];
memset(outputs, 0, sizeof(outputs));
outputs[0].want_float = 1;
outputs[0].index = 0;
ret = amlnn_outputs_get(ctx, 1, outputs);
```

amlnn_destroy

The `amlnn_destroy` function releases the passed-in `amlnn` context and its related resources.

API	amlnn_destroy
Function	Destroys the <code>amlnn</code> context object and its related resources
Parameters	void* context: Context handle pointer
Return Value	int error code (see AMLNN API Error Return Codes)

```
ret = amlnn_destroy(ctx);
```

amlnn_alloc_dma_mem

For ADLA NPU only

When the user needs the NPU to allocate DMA memory internally, `amlnn_alloc_dma_mem` allocates the user-specified memory size and type. Types include cache type, access type, buffer type, and I/O type. Memory size is specified through an `amlnn_dma_memory` structure object, and memory type is specified through an `amlnn_dma_config` structure object. Before calling the function, create and configure `amlnn_dma_memory` and `amlnn_dma_config` objects.

DMA Memory Allocation Notes:

- **Virtual address mode:** The `amlnn_alloc_dma_mem` function must be called for allocation. User-created virtual address mapping or allocation is currently **not supported**.
- **Physical address mode:** The `amlnn_alloc_dma_mem` function does not need to be called. The user provides the address, but must ensure that the physical start address satisfies the **4K** alignment requirement.

DMA Memory Space Alignment Requirements:

To be compatible with hardware memory-access stride requirements, distinguish between "allocated space" and "used space" when using DMA memory:

- **Allocation criterion:** The actual size of requested DMA memory must be based on **size** in the `amlnn_tensor_attr` structure. If the allocated size is smaller than **size**, hardware out-of-bounds access may occur, causing system crashes or data errors.
- **Usage criterion:** During logical processing or data filling, the valid data range should be based on **size** in the `amlnn_tensor_attr` structure.

API	amlnn_alloc_dma_mem
Function	Allocates DMA memory
Parameters	void* context: Context handle pointer

API	amlnn_alloc_dma_mem
	amlnn_dma_memory* dma_memory: DMA memory information structure object used to describe one DMA buffer
	amlnn_dma_config* dma_config: DMA memory configuration parameters
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
amlnn_dma_config dma_config_input;
memset(&dma_config_input, 0, sizeof(amlnn_dma_config));
dma_config_input.cacheable = 1;
dma_config_input.access_mode = AMLNN_EXT_MEM_ACCESS_READ_WRITE;
dma_config_input.buffer_type = AMLNN_EXT_BUFFER_VIRTUAL_ADDR;
dma_config_input.io_type = AMLNN_EXT_IO_INPUT;

amlnn_dma_memory dma_memory_input[1];
memset(dma_memory_input, 0, sizeof(dma_memory_input));
dma_memory_input[0].index = 0;
dma_memory_input[0].size = input_attrs[0].size;
ret = amlnn_alloc_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);
```

```
amlnn_dma_config dma_config_output;
memset(&dma_config_output, 0, sizeof(amlnn_dma_config));
dma_config_output.cacheable = 1;
dma_config_output.access_mode = AMLNN_EXT_MEM_ACCESS_READ_WRITE;
dma_config_output.buffer_type = AMLNN_EXT_BUFFER_VIRTUAL_ADDR;
dma_config_output.io_type = AMLNN_EXT_IO_OUTPUT;

amlnn_dma_memory dma_memory_output[1];
memset(dma_memory_output, 0, sizeof(dma_memory_output));
dma_memory_output[0].index = 0;
dma_memory_output[0].size = output_attrs[0].size;
ret = amlnn_alloc_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);
```

amlnn_bind_dma_mem

For ADLA NPU only

After DMA memory is allocated and before `amlnn_run` is called, `amlnn_bind_dma_mem` can bind the DMA memory to the NPU.

API	amlnn_bind_dma_mem
Function	Binds DMA memory for NPU use
Parameters	void* context: Context handle pointer
	amlnn_dma_memory* dma_memory: DMA memory information structure object used to describe one DMA buffer

API	amlnn_bind_dma_mem
	amlnn_dma_config* dma_config: DMA memory configuration parameters
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
ret = amlnn_bind_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);
ret = amlnn_bind_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);
```

amlnn_unbind_dma_mem

For ADLA NPU only

After DMA memory is used, `amlnn_unbind_dma_mem` can unbind all DMA memory previously bound to the NPU.

API	amlnn_unbind_dma_mem
Function	Unbinds DMA memory
Parameters	void* context: Context handle pointer
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
ret = amlnn_unbind_dma_mem(ctx);
```

amlnn_free_dma_mem

For ADLA NPU only

The `amlnn_free_dma_mem` function releases DMA memory created by `amlnn_alloc_dma_mem`.

API	amlnn_free_dma_mem
Function	Releases DMA memory
Parameters	void* context: Context handle pointer
	amlnn_dma_memory* dma_memory: DMA memory information structure object used to describe one DMA buffer
	amlnn_dma_config* dma_config: DMA memory configuration parameters
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
ret = amlnn_free_dma_mem(ctx, &dma_memory_input[0], &dma_config_input);
ret = amlnn_free_dma_mem(ctx, &dma_memory_output[0], &dma_config_output);
```

amlnn_set_softop_opt

For ADLA NPU only

The `amlnn_set_softop_opt` function configures acceleration options for software operators.

API	amlnn_set_softop_opt
Function	Configures software operator acceleration options
Parameters	void* context: Context handle pointer
	amlnn_softop_opt_request requests: Software operator acceleration configuration array; each array element is an <code>amlnn_softop_opt_request</code> structure object
	uint32_t request_num: Number of configuration array elements
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
amlnn_softop_opt_request softop_opt_requests[] =
{
    {
        .softop_type = AMLNN_Unknown,           // Operator type to set
        .acc_type = AMLNN_SOFTOP_ACC_OPENMP,    // Select OpenMP acceleration
    },
};
ret = amlnn_set_softop_opt(ctx, softop_opt_requests, sizeof(softop_opt_requests)
/ sizeof(amlnn_softop_opt_request));
```

amlnn_set_input_shapes

For models with dynamic-shape inputs, the currently used input shape must be specified before inference. This API receives the number of inputs and an `amlnn_tensor_attr` array containing each input shape and corresponding data layout information. For each `amlnn_tensor_attr` object, the index, name, shape (`dims`), and memory layout information (`fmt`) must be filled in; other members do not need to be set. Before using `amlnn_set_input_shapes`, use `amlnn_query` to query the number of supported input shapes and the dynamic shape list. The input data shape must be in the model's supported input shape list. On the first run or whenever switching to a new input shape, call this API to set the new shape; otherwise, repeated calls are not required.

API	amlnn_set_input_shapes
Function	Sets the input shape currently used by the model
Parameters	void* context: Context handle pointer

API	amlnn_set_input_shapes
	uint32_t n_inputs: Number of input tensors
	amlnn_tensor_attr *: Pointer to an input tensor attribute array that passes shape information for all inputs. The user must set the <code>index</code> , <code>name</code> , <code>dims</code> , <code>fmt</code> , and <code>n_dims</code> members in each input attribute structure; other members do not need to be set
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    for (uint32_t j = 0; j < input_attrs[i].n_dims; ++j)
    {
        input_attrs[i].dims[j] = shape_range[i].dyn_range[s][j];
    }
}

ret = amlnn_set_input_shapes(ctx, io_num.n_input, input_attrs);
```

amlnn_set_core_mask

For ADLA NPU only

The `amlnn_set_core_mask` function specifies the NPU core to work on.

API	amlnn_set_core_mask
Function	Specifies the NPU core to work on
Parameters	void* context: Context handle pointer
	amlnn_core_mask_config* core_mask_config: NPU core mask configuration information
Return Value	int error code (see AMLNN API Error Return Codes)

Example code:

```
amlnn_core_mask_config core_mask_config;
memset(&core_mask_config, 0, sizeof(amlnn_core_mask_config));
core_mask_config.requested_mask = 0x1; // Use the first NPU core
core_mask_config.policy = AMLNN_CORE_MASK_BEST_EFFORT;
ret = amlnn_set_core_mask(ctx, &core_mask_config);
```

4 Error Return Codes

The return-value error codes of AMLNN API functions are defined in the table below:

Return Value	Description
AMLNN_SUCCESS (0)	Execution succeeded
AMLNN_ERR_FAIL (-1)	Execution failed (generic error)
AMLNN_ERR_TIMEOUT (-2)	Execution timed out
AMLNN_ERR_DEVICE_UNAVAILABLE (-3)	Device unavailable
AMLNN_ERR_MALLOC_FAIL (-4)	Memory allocation failed
AMLNN_ERR_PARAM_INVALID (-5)	Invalid parameter
AMLNN_ERR_MODEL_INVALID (-6)	Invalid model
AMLNN_ERR_CTX_INVALID (-7)	Invalid context
AMLNN_ERR_INPUT_INVALID (-8)	Invalid input
AMLNN_ERR_OUTPUT_INVALID (-9)	Invalid output
AMLNN_ERR_NOT_SUPPORTED (-10)	Feature not supported
AMLNN_ERR_NOT_READY (-11)	Result not ready
AMLNN_ERR_NOT_MATCH (-12)	Not matched